Lists and tuples

Data structures

- Collections of data values or objects that contain different data types
- Data professionals use data structures to store, access, organize, and categorize their data with speed and efficiency. Knowing which data structures fit your specific task is a key part of data work, and will help streamline your analysis.

List

- A data structure that helps store and manipulate an ordered collection of items

Lists	Strings
Data structure	Data type
Allow duplicate elements	Allow duplicate elements
Allow indexing and slicing	Allow indexing and slicing
sequences of elements	sequences of characters
Mutable	immutable

Mutability

- The ability to change the internal state of a data structure

Modify the contents of a list

insert()

- Function that takes an index as the first parameter and an element as the second parameter, then inserts the element into a list at the given index

remove()

- A method that removes an element from a list

pop()

- A method that extracts an element from a list by removing it a given index

Reference guide: Lists

You've been learning that lists are important data structures in Python. A list is a data structure that helps store and manipulate an ordered collection of items. These items can be of any data type such as integers, floats, strings, and even other lists. Because they are so versatile, data professionals and all Python programmers use lists every day, so it's important to be familiar with how they work. This reading is a reference guide for lists designed to help you as you learn Python.

Save this course item

You may want to save a copy of this guide for future reference. You can use it as a resource for additional practice or in your future professional projects. To access a downloadable version of this course item, click the following link and select "Use Template."

Reference guide: Lists

OR

If you don't have a Google account, you can download the item directly from the attachment below.

Reference guide_ ListsDOCX File

Create a list

There are two main ways to create lists in Python:

- Square brackets: **[]**
- The list function: **list()**

When instantiating a list using brackets, separate each element with a comma.

For example, the following code creates a list of strings:

```
list_a = ['olive', 'palm', 'coconut']
print(list_a)
```

You can also create a list of integers:

```
list_b = [8, 6, 7, 5, 3, 0, 8]
```

```
print(list_b)
```

Or a list of mixed data types:

```
list_c = ['Abidjan', 14.2, [1, 2, None], 'Zagreb']
print(list_c)
```

To create an empty list, use empty brackets or the **list()** function:

```
empty_list_1 = []
empty_list_2 = list()
```

Indexing and slicing

Just as with strings, you can access elements in a list using indexing and slicing. The first element of a list has index zero, the second element has index one, and so on. Use square brackets to index:

```
phrase = ['Astra', 'inclinant', 'sed', 'non', 'obligant']
print(phrase[1])
```

You can also use negative indices to access items from the end of a list:

```
phrase = ['Astra', 'inclinant', 'sed', 'non', 'obligant']
print(phrase[-1])
```

Notice that this code returned a sublist containing the elements at indices one, two, and three of phrase. The ending index of the slice is not included.

Omitting the starting index in a slice implies an index of zero, and omitting the ending index implies an index of **len(my_list)**:

```
phrase = ['Astra', 'inclinant', 'sed', 'non', 'obligant']
print(phrase[:3]) # this is called slicing
print(phrase[3:])
```

List mutability

Lists are mutable, which means that you can change their contents after they are created. You can change an individual item in a list by specifying its index and assigning a new value to it. For example:

```
my_list = ['Macduff', 'Malcolm', 'Duncan', 'Banquo']
my_list[2] = 'Macbeth'
print(my_list)
```

You can even change a slice of a list using the same logic. The slice can be of any length. The elements in the new list will be inserted in place of the indicated slice:

```
my_list = ['Macduff', 'Malcolm', 'Macbeth', 'Banquo']
my_list[1:3] = [1, 2, 3, 4]
print(my_list)
['Macduff', 1, 2, 3, 4, 'Banquo']
```

List operations

Lists can be combined using the addition operator (**+**):

```
num_list = [1, 2, 3]
char_list = ['a', 'b', 'c']
num_list + char_list

[1, 2, 3, 'a', 'b', 'c']
```

They can also be multiplied using the multiplication operator (*****):

```
list_a = ['a', 'b', 'c']
list_a * 2

['a', 'b', 'c', 'a', 'b', 'c']
```

But they cannot be subtracted or divided.

You can check whether a value is contained in a list by using the **in** operator:

```
num_list = [2, 4, 6]
print(5 in num_list)
print(5 not in num_list)
```

List methods

Lists are a core Python class. As you've learned, classes package data together with tools to work with it. Methods are functions that belong to a class. Lists have a number of built-in methods that are very useful.

append()

Add an element to the end of a list:

```
my_list = [0, 1, 1, 2, 3]
variable = 5
my_list.append(variable)
print(my_list)

[0, 1, 1, 2, 3, 5]
```

insert()

Insert an element at a given position:

```
my_list = ['a', 'b', 'd']
my_list.insert(2, 'c')
print(my_list)

['a', 'b', 'c', 'd']
```

remove()

Remove the first occurrence of an item:

```
my_list = ['a', 'b', 'd', 'a']
my_list.remove('a')
```

```
print(my_list)

['b', 'd', 'a']
```

pop()

Remove the item at the given position in the list, and return it. If no index is specified, **pop()** removes and returns the last item in the list:

```
my_list = ['a', 'b', 'c']
print(my_list.pop())
print(my_list)

#output
c
['a', 'b']
```

clear()

Remove all items:

```
my_list = ['a', 'b', 'c']
my_list.clear()
print(my_list)
```

index()

Return the index of the first occurrence of an item in the list:

```
my_list = ['a', 'b', 'c', 'a']
my_list.index('a')

#output
0
```

count()

Return the number of times an item occurs in the list:

```
my_list = ['a', 'b', 'c', 'a']
my_list.count('a')

#output
2
```

sort()

Sorts the list ascending by default. You can also make a function to decide the sorting criteria:

```
char_list = ['b', 'c', 'a']
num_list = [2, 3, 1]
char_list.sort()
num_list.sort(reverse=True)
print(char_list)
print(num_list)

#output
['a', 'b', 'c']
[3, 2, 1]
```

Additional resources

- For more information about lists, refer to An Informal Introduction to Python: Lists.
- For more list methods, refer to Data Structures: More on Lists

Introduction to tuples

Tuple

- An immutable sequence that can contain elements of any data type
- Tuples are expressed with parentheses or the tuple() function

Tuple() function

- Function that transforms input into tuples

```
full_name = ["Dinh", "Xuan", "Khuong"]
full_name = tuple(full_name)
print(full_name)
```

> When a function returns more than two values, it’s actually returning a tuple

```
def to_dollars_cents(price):
    dollars = int(price//1)
    cents = round(price % 1 * 100)

    return dollars, cents
```

Compare lists, strings, and tuples

You’ve now learned about some of Python’s core iterable sequence data structures, including strings, lists, and tuples. These structures share many similarities, but there are some key differences between them. Data professionals must often decide which data structures work best to solve a particular problem, so understanding the relationship between these classes can help you make informed decisions in your work. This reading is a guide to the similarities and differences between strings, lists, and tuples.

Strings

Syntax/instantiation

Note: The following code block is not interactive.

- Single, double, or triple quotes:

```
empty_str = ''
my_string1 = 'minerals'
my_string2 = "martin"
```

```
my_string3 = """
marathon
golfcart
"""
```

Note: Using triple quotes to write a string over multiple lines will insert newlines (**\n**).

```
my_string3 = """
marathon
golfcart
"""

my_string3

#output

marathon
golfcart
```

• The
str() function can be used for instantiation and conversion.

Note: The following code block is not interactive.

```
 empty_str = str()
my_string = str(125)
```

Content

- Strings can contain any character—letters, numbers, punctuation marks, spaces—but everything between the opening and closing quotation marks is part of the same single string.

Mutability

- Strings are **immutable**. This means that once a string is created, it cannot be modified. Any operation that appears to modify a string actually creates a new string object.

Usage

- Strings are most commonly used to represent text data.

Methods

The Python **string** class comes packed with many useful methods to manipulate the data contained in strings. For more information on these methods, refer to Common String Operations in the Python documentation.

Lists

Syntax/instantiation

- Brackets, with each element separated by a comma:

Note: The following code block is not interactive.

```
empty_list = []
my_list = [1, 2, 3, 4, 5]
```

• The
list() function can be used for instantiation and conversion. Note that this function only works on iterable data types.

Mutability

- Lists are **mutable**. This means that they can be modified after they are created.

```
num_list = [1, 2, 3]
num_list[0] = 5446
print(num_list)
```

Usage

- Lists are very versatile and therefore are used in numerous cases. Some common ones are:
    - Storing collections of related items

- Storing collections of items that you want to iterate over: Because lists are ordered, you can easily iterate over their elements using a for loop or list comprehension.
- Sorting and searching: Lists can be sorted and searched, making them useful for tasks such as finding the minimum or maximum value in a list or sorting a list of items alphabetically.
- Modifying existing data: Because lists are mutable, they are useful for situations in which you know you'll need to modify your data.
- Storing results: Lists can be used to store the results of a computation or a series of operations, making them useful in many different programming tasks.

Methods

- You can find methods for the Python list class in More on Lists in the Python documentation.

Tuples

Syntax/instantiation

- Parentheses, with each element separated by a comma:

Note: The following code block is not interactive.

```
empty_tuple = ()
my_tuple = (1, 'z')
```

Note: When using parentheses to declare a tuple with just a single element, you must use a trailing comma.

```
test1 = (1)
test2 = (2,)

print(type(test1))
print(type(test2))


#output
```

```
<class 'int'>
<class 'tuple'>
```

• No parentheses, but each element followed by a comma (even if there’s only one element):

```
tuple1 = 1,
tuple2 = 2, 3

print(type(tuple1))
print(type(tuple2))
#output
<class 'tuple'>
<class 'tuple'>
```

- The **tuple()** function can be used for instantiation, and for conversion of iterable data types.

Note: The following code block is not interactive.

```
empty_tuple = tuple()
my_tuple = tuple([1, 'z'])
```

Content

- Tuples can contain any data type, and in any combination. So, a single tuple can contain strings, integers, floats, lists, dictionaries, and other tuples.

Note: The following code block is not interactive.

```
my_tuple = (1871, 'all', 'mimsy', ('were', 'the'), ['borogroves'])
```

Mutability

- Tuples are **immutable**. This means that once a tuple is created, it cannot be modified.

Usage

- Common uses of tuples include:
  - Returning multiple values from a function
  - Packing and unpacking sequences: You can use tuples to assign multiple values in a single line of code.
  - Dictionary keys: Because tuples are immutable, they can be used as dictionary keys, whereas lists cannot. (You'll learn more about dictionaries later.)
  - Data integrity: Due to their immutable nature, tuples are a more secure way of storing data because they safeguard against accidental changes.

Methods

- Because tuples are built for data security, Python has only two methods that can be used on them:
  - **count()** returns the number of times a specified value occurs in the tuple.
  - **index()** searches the tuple for a specified value and returns the index of the first occurrence of the value.

Key takeaways

Strings, lists, and tuples are all iterable sequential data structures that share many similarities. They also have fundamental differences that you should be aware of so you can make effective choices in your work as a data professional. When selecting a data structure, consider its manner of instantiation, content, mutability, and the use case.

Resources:

- For more information about strings, refer to the Introduction to Python strings documentation.
- For more information about lists, refer to the Introduction to Python lists documentation.
- For more information about tuples, refer to the Python Standard Data Types tuples documentation

More with loops, lists, and tuples

List comprehension

- Formulaic creation of a new list based on the values in an existing list

```
pips_from_loop = []
for domino in dominoes:
    pips_from_loop.append(domino[0] + domino[1])

#same as
pips_from_list_comp = [domino[0] + domino[1] for domino in dominoes]
```

zip(), enumerate(), and list comprehension

You've learned much about iterable objects such as strings, lists, and tuples, and soon you'll learn more. These objects comprise many of Python's core data structures and, as a data professional, you'll work with them constantly. While working in Python, you'll often need to perform the same tasks and operations many times. This reading will introduce you to three time-saving tools: **zip()**, **enumerate()**, and list comprehension.

zip()

The zip() function is a built-in Python function that does what the name implies: It performs an element-wise combination of sequences.

The function returns an **iterator** that produces tuples containing elements from each of the input sequences. An iterator is an object that enables processing of a collection of items one at a time without needing to assemble the entire collection at once. Use an iterator with loops or other iterable functions such as **list()** or **tuple()**. Here's an example:

```
cities = ['Paris', 'Lagos', 'Mumbai']
countries = ['France', 'Nigeria', 'India']
places = zip(cities, countries)
```

```
print(places)
print(list(places))

##
<zip object at 0x7f28130898c8>
[('Paris', 'France'), ('Lagos', 'Nigeria'), ('Mumbai', 'India')]
```

Notice that, in this case, the **list()** function is used to generate a list of tuples from the iterator object. Here are a few things to keep in mind when using the **zip()** function.

- It works with two or more iterable objects. The given example zips two sequences, but the **zip()** function will accept more sequences and apply the same logic.
- If the input objects are of unequal length, the resulting iterator will be the same length as the shortest input.
- If you give it only one iterable object as an argument, the function will return an iterator that produces tuples containing only one element from that iterable at a time.

Unzipping

You can also unzip an object with the * operator. Here's the syntax:

Notice that, in this case, the **list()** function is used to generate a list of tuples from the iterator object. Here are a few things to keep in mind when using the **zip()** function.

- It works with two or more iterable objects. The given example zips two sequences, but the **zip()** function will accept more sequences and apply the same logic.
- If the input objects are of unequal length, the resulting iterator will be the same length as the shortest input.
- If you give it only one iterable object as an argument, the function will return an iterator that produces tuples containing only one element from that iterable at a time.

Unzipping

You can also unzip an object with the * operator. Here's the syntax:

```
scientists = [('Nikola', 'Tesla'), ('Charles', 'Darwin'), ('Marie', 'Curie')]
given_names, surnames = zip(*scientists)
print(given_names)
print(surnames)


##
('Nikola', 'Charles', 'Marie')
('Tesla', 'Darwin', 'Curie')
```

Note that this operation unpacks the tuples in the original list element-wise into two tuples, thus separating the data into different variables that can be manipulated further.

enumerate()

The enumerate() function is another built-in Python function that allows you to iterate over a sequence while keeping track of each element's index. Similar to **zip()**, it returns an iterator that produces pairs of indices and elements. Here's an example:

```
letters = ['a', 'b', 'c']
for index, letter in enumerate(letters):
  print(index, letter)

 ##
0 a
1 b
2 c
```

Note that the default starting index is zero, but you can assign it to whatever you want when you call the **enumerate()** function. For example:

```
letters = ['a', 'b', 'c']
for index, letter in enumerate(letters, 2):
  print(index, letter)
###
2 a
```

```
3 b
4 c
```

In this case, the number two was passed as an argument to the function, and the first element of the resulting iterator had an index of two. The **enumerate()** function is useful when an element's place in a sequence must be used to determine how the element should be handled in an operation.

List comprehension

One of the most useful tools in Python is list comprehension. List comprehension is a concise and efficient way to create a new list based on the values in an existing iterable object. List comprehensions take the following form:

my_list = [expression for element in iterable if condition]

In this syntax:

- **expression** refers to an operation or what you want to do with each element in the iterable sequence.
- **element** is the variable name that you assign to represent each item in the iterable sequence.
- **iterable** is the iterable sequence.
- **condition** is any expression that evaluates to **True** or **False**. This element is optional and is used to filter elements of the iterable sequence.

Here are some examples of list comprehensions:

This list comprehension adds 10 to each number in the list:

```
numbers = [1, 2, 3, 4, 5]
new_list = [x + 10 for x in numbers]
print(new_list)

##
[11, 12, 13, 14, 15]
```

In the preceding example, **x + 10** is the expression, **x** is the element, and **numbers** is the iterable sequence. There is no condition.

This next list comprehension extracts the first and last letter of each word as a tuple, but only if the word is more than five letters long.

```
words = ['Emotan', 'Amina', 'Ibeno', 'Sankwala']
new_list = [(word[0], word[-1]) for word in words if len(word) > 5]
print(new_list)



##
[('E', 'n'), ('S', 'a')]
```

Note that multiple operations can be performed in the expression component of the list comprehension to result in a list of tuples. This example also makes use of a condition to filter out words that are not more than five letters long.

Key takeaways

zip(), **enumerate()**, and list comprehension make code more efficient by reducing the need to rely on loops to process data and simplifying working with iterables. Understanding these common tools will save you time and make your process much more dynamic when manipulating data.